

Etape 2 livree, et pourquoi 1b et 3 ne le sont pas

Force Tracker — 12/09/2026, ft -v1194. Sixieme document de la serie (DOUANE -> ARCHI -> PLAN -> PHASE0A-ETAPE1A -> VALIDATION-IPHONE -> celui-ci). Michel a valide la phase 0a **sur iPhone** et donne le feu vert pour **1b, 2 et 3**, meme methode que 1a. **Une seule des trois est livree**, et c'est sa propre consigne qui l'a decide.

LA CONSIGNE QUI DECIDE DE CETTE VERSION

« Si une regression ou une divergence reelle apparait pendant l'extraction, **mesure-la et arrete-toi** avant de la 'corriger au passage'. »

La divergence est apparue AVANT la premiere ligne de code — et elle ne porte pas sur le code, elle porte sur le **perimetre** ecrit dans le plan.

1. Le perimetre : ce que le plan annonce, ce que la mesure donne

etape	perimetre du plan	perimetre MESURE	fait ?
1b — la « forme aliment »	5 sites	19 lignes portent portionWeightG a elles seules, en 3 formes	non
2 — le pour-100 g derive	3 sites	3 — <i>le seul juste</i>	livree
3 — « quantite utilisable ? »	6 sites	7 ecritures en 2 formes (5 + 2), + 9 lignes de la moitie PORTIONS, jamais comptee	non

Et ce ne sont pas des ecarts de comptage : les DEFAUTS divergent d'un site a l'autre. C'est ce qui interdit une extraction mecanique — choisir une valeur ne serait pas un rangement, ce serait une **decision** qui change des lignes deja enregistrees :

champ	ce qu'on trouve selon le site
q	tantot 0 , tantot null
portionWeightG	0 , null , ou la cle est absente
origine	null , 'utilisateur', ou 'reprise'

L'instantane tools/instantane_1b23.js **fige ces trois ecarts expres** : il est ce qui empeche de les harmoniser « au passage » sans s'en apercevoir.

2. Ce qui est livre : un seul propriétaire du pour-100 g derive

Trois endroits retapaient la meme algebre — totaux x 100 / masse, macro par macro, arrondie a la decimale.

```
// AVANT – la meme algebre, retapee a trois endroits

// _provFood, branche GRAMMES
const f=100/q;
p.per100={kcal:_per100d1((+vals.kcal||0)*f),
  prot:_per100d1((+vals.prot||0)*f),
  carbs:_per100d1((+vals.carbs||0)*f),
  fat:_per100d1((+vals.fat||0)*f)};

// saveEditFood
const f=100/masse;
e.per100={kcal:_per100d1((+e.kcal||0)*f), prot:_per100d1((+e.prot||0)*f),
  carbs:_per100d1((+e.carbs||0)*f), fat:_per100d1((+e.fat||0)*f)};

// _provFood, branche PORTIONS
const f=100/masse;
p.per100={kcal:_per100d1((+vals.kcal||0)*f),
  prot:_per100d1((+vals.prot||0)*f),
  carbs:_per100d1((+vals.carbs||0)*f),
  fat:_per100d1((+vals.fat||0)*f)};

// APRES – un seul propriétaire, et il sait DIRE QU'IL NE SAIT PAS

function _per100Derive(vals, masse){
  const m=+masse||0;
  if(!(m>0)) return null; // <- ce null EFFACE un pour-100 g orphelin (R29)
  const f=100/m;
  return {kcal:_per100d1((+vals.kcal||0)*f), prot:_per100d1((+vals.prot||0)*f),
    carbs:_per100d1((+vals.carbs||0)*f), fat:_per100d1((+vals.fat||0)*f)};
}

// les 3 appelants
p.per100=_per100Derive(vals, q); // grammes
if(!p.per100){ const d=_per100Derive(vals, masse); if(d) p.per100=d; } // portions
const d=_per100Derive(e, (+e.q||0)*(+e.portionWeightG||0)); // edition
if(d) e.per100=d; else if(e.per100) delete e.per100;
```

LA DUPLICATION AVAIT DEJA COUTE, SUR CES LIGNES-LA

En **ft-v1188**, le passage de `Math.round` a `_per100d1` a du etre pose sur **deux** d'entre elles — apres l'avoir ete sur **7 autres portes** en **ft-v1170**. C'est la famille **\$59** du depot (*la porte jumelle*), a sa 8^e occurrence.

La question n'etait pas de savoir si la 3^e copie serait oubliee, mais quand.

3. Le point de conception : ce qu'on factorise est l'INTENTION, pas la ressemblance

A six lignes du nouveau propriétaire vit une fonction qui porte **exactement la meme algebre**. On est tente de l'absorber. **Elle est lailsee dehors.**

```
// LAISSEE DEHORS, EXPRES – meme algebre, autre metier

function _per100SuitLaPortion(av){ // elle VERIFIE : « ce pour-100 g
  if(!av.per100) return true; // venait-il d'une portion ? »
  const m=(+av.q||0)*(+av.pw||0); // ... et non « quelle est sa valeur ? »
  if(!(m>0)) return false;
  const f=100/m, ok=(a,b)=>Math.abs((+a||0)-(+b||0))<=0.6; // <- sa tolerance
  return ok(av.per100.kcal,(+av.kcal||0)*f) && ...
}
```

Elle **verifie** (« ce pour-100 g venait-il d'une portion ? », a 0,6 pres), elle ne **derive** pas. La fondre dans le propriétaire changerait **son metier**, pas son code. Un temoin fige qu'elle est toujours la, avec sa tolerance.

ET LE PROPRIETAIRE SAIT DIRE QU'IL NE SAIT PAS

Masse nulle, negative, illisible -> il rend `null`. **Ce null n'est pas decoratif** : c'est lui qui, dans `saveEditFood`, **EFFACE** un pour-100 g devenu orphelin quand la personne retire le poids de sa portion. **R29** — un `null` ne se remplace jamais par un default.

4. L'etape 3 en detail : le plan comptait la moitie de la question

Le « 6 » du plan etait presque juste sur la regle stricte : elle est ecrite 7 fois, en 2 formes.

forme	combien	ce qu'elle accepte
+q>0 && (!u u==='g')	5	les grammes seuls (et l'unite absente)
... u==='portion'	2	les grammes et les portions

CE QUE LE PLAN A RATE : LA MOITIE PORTIONS

u==='portion' apparait **9 fois** de plus, sur des lignes qui decident elles aussi de ce qu'on fait de la quantite. **Elle a ete ajoutee en ft-v1183 / ft-v1186 et jamais reintegree a l'inventaire.**

Total reel : **16 decisions** sur l'unite, pas 6.

ET J'AI FAILLI LIVRER ICI LE MEME GENRE DE CHIFFRE QUE CELUI QUE JE REPROCHE AU PLAN

Ma premiere redaction annoncait « ~17 sites, au moins 3 ecritures de la regle », en citant _provFood @1232 contre @1311 comme **deux ecritures non equivalentes.**

Relu ligne a ligne avant de pousser : c'est faux. @1311 ne pose pas la meme question — elle interroge _afRef (l'etat de l'ecran), pas _afSrc (la provenance de l'aliment) — et son === strict y est **necessaire** : _afRef . u vaut ' ' dans l'etat « portions », donc un ! u permissif ferait tomber une portion dans la branche grammes.

Un chiffre rond se verifie ligne a ligne — surtout quand il sert a demontrer qu'un autre chiffre etait faux.

5. Pourquoi le compteur s'est trompé (nouvelle famille §63)

```
// POURQUOI LE COMPTEUR S'EST TROMPE — addFoodEntry, la porte la plus utilisee

const _e=Object.assign({date:_journalJourActif(), meal:_afMeal, name:name.slice(0,80),
                        kcal, prot, carbs, fat, ts:Date.now()}, // <- raccourci ES6
                        _provFood({kcal,prot,carbs,fat}));

S.foodLog.push(_e);

// Il n'y a PAS UN SEUL « kcal: » dans cette ligne.
// Un motif /kcal\s*/ ne la voit pas — et il ne signale rien, puisqu'il trouve les autres.
```

Meme cause pour l'export CSV de setup.js : **13 colonnes** aux noms **français** (quantite, unite, portion_label...), avec sa liste figée a part. Aucun motif pensé pour le schema interne ne peut la voir.

LA FAMILLE, ECRITE DANS `BUGS.md` §63

Un motif de recherche qui suppose une SYNTAXE ne compte pas les endroits : il compte les endroits écrits comme on les imaginait.

Pourquoi c'est couteux : le chiffre sert ensuite a **dimensionner un chantier**. Une extraction « de 6 sites » qu'on decouvre a 16 en cours de route se termine de deux facons, toutes deux mauvaises — on livre a moitié, ou on **elargit en silence** un perimetre que personne n'a valide. *Un inventaire faux ne se manifeste pas comme une erreur : il se manifeste comme un chantier qui deborde.*

Ce qui l'attrape : compter **deux fois, autrement** (par motif de texte **et** par ce que la fonction FAIT — ici « qui écrit dans S.foodLog ? », qui trouve les **3** ecrivains quelle que soit leur syntaxe) ; croiser avec les appelants (*la porte principale d'un ecran ne peut pas etre absente d'un inventaire de cet ecran*) ; et **dire le chiffre avec sa methode** — « 7 lignes correspondent au motif X » est verifiable, « il y a 7 sites » ne l'est pas.

Soeur de §58 (verifier la fonction n'est pas verifier l'appel) et de §61 (un outil de mesure tronque ressemble a un code sans default) : les trois disent la meme chose a trois endroits — **l'instrument fait partie de la mesure.**

6. Comment on sait que rien n'a bougé

preuve	resultat
Instantane tools/instantane_1b23.js (11 sondes), avant / apres	identique octet pour octet , meme sha256 ace2a744dc89e6ec — les sondes de 1b et 3 comprises , donc l'extraction n'a pas deborde
Passe parcours (bloc CCXCI, +15 temoins)	3564 / 3564 — et le total predict etait 3549 + 15 = 3564 : il colle, donc la passe n'est pas tronquee (§61)
Calculs / muscles / croises / dates	339 / 241 / 50 / 9 , tous verts
Donnees classees face a Milo	aucun trou nouveau (les 2 connus restent inchangés)
Controle negatif	8 mutations, TOUTES mordent , chacune sur son temoin
Deploiement (R18)	run #1091 , success a 08:02:02 UTC

UNE MUTATION A TUE MA SONDE AU LIEU DE LA FAIRE ROUGIR

Le temoin des 4 macros ecrivait `Object.keys(_per100Derive(...))`. Avec le propriétaire mute pour rendre null, ca **leve** — l'evaluaté entier est rejete, et le **bloc disparaît de la passe sans qu'elle rougisce.**

Rendu defensif, la mutation fait désormais **7 rouges nommes**. *Un temoin qui MEURT ressemble a un temoin qui passe* — c'est §61 en miniature, et la raison est écrite a l'endroit exact.

7. Ce qui attend une decision de Michel

Deux questions, et elles ne sont pas techniques. Elles sont ecrites dans docs/JOURNAL - DE - TEST .md pour ne pas disparaitre avec la session (R27).

question	ce qui en depend
1. 1b et 3 se font-elles sur leur perimetre reel (un chantier ~3x plus gros que le plan), ou re-decoupees en morceaux plus petits ?	le decoupage du chantier, et le nombre de versions avant le hub
2. Les defauts divergents s'harmonisent-ils ?	si oui , c'est une decision produit : elle change des lignes deja enregistrees. Si non , la « forme aliment » unique devra les transporter — ce qui limite beaucoup l'interet de l'etape

Hors perimetre, et ca le reste

sujet	etat
Hub (etape 4) et douane (etape 5)	apres 1b et 3 — consigne explicite de Michel
S. savedFoods perdu entre deux onglets	ouvert . Le correctif evident est faux : une union par nom ferait ressusciter une etoile retiree . Decision produit
L'ecart 48,3 vs 48 sur la meme fiche	ouvert . Corriger changerait une valeur enregistree — ce n'est pas une extraction
Historique abime, migrations, Milo, seances	non touches

LIMITE DITE PLUTOT QUE TUE

Le proxy de ce conteneur refuse `github.io` (403), donc **je ne peux pas lire le sw.js reellement servi**. La verification s'arrete a l'API GitHub : *le run est vert, l'app affichant ft-v1194 reste a confirmer par Michel dans « A propos »*.

Document genere depuis le code servi — **tous les decomptes cites sont recomptes a chaque generation** (1 proprietaire, 4 appels, 5 + 2 ecritures de la regle, 9 lignes de la moitie portions, 13 colonnes CSV, 3 ecrivains). *Un document qui cite un decompte sans le verifier refait exactement l'erreur qu'il raconte*. Source : `tools/gen_etape2_perimetre_pdf.py`.